Europa-Universität Viadrina

Faculty of Business Administration and Economics

Chair of Supply Chain Management

Project Report

Submitted for the Course

Management Science

Project 3: Solving the Multi-Level Capacitated Lot Sizing Problem with Lot Streaming

Submitted to:	Prof. Christian Almeder
by	Long Dinh
Matriculation no.	159862
Date	June 21, 2026

Abstract

This project investigates the Multi-Level Capacitated Lot Sizing Problem (MLCLSP) with lot streaming. The objective is to determine production quantities and setup decisions over a finite planning horizon while satisfying demand, respecting machine capacities, and synchronizing material flows within a multi-level bill-of-materials structure. The mathematical formulation proposed in the reference paper is implemented in Python using Gurobi. Computational experiments are conducted on benchmark datasets.

Table of Contents

Figures III

Tables III

Chapter 1: Introduction i

Chapter 2: Mathematical Model ii

2.1 Sets and Indices ii

2.2 Parameters ii

2.3 Decision Variables iii

2.4 Objective Function iv

2.5 Constraints iv

Chapter 3: Implementation viii

3.1 Benchmark instances viii

3.2 Data Parsing ix

3.2.1 Section Identification x

3.2.2 Mapping Between File Sections and Data Structures x

3.2.3 Parser Output xi

3.3 Data Preprocessing and Set Construction xii

3.3.1 Capacity Normalization xii

3.3.2 Parameter and Set Construction xii

3.4 Model Implementation xiii

3.4.1 Decision Variables xiii

3.4.2 Objective Function xiv

3.4.3 Implementation of Problem Constraints xiv

Chapter 4: Computational Results xvi

4.1 Computational Results for Instance A xvi

4.2 Computational Results for Instance B..... xix

4.2.1 Analysis of infeasibility..... xix

4.2.2 Remedies for Infeasibility xx

4.3 Computational Results for Instances C and D xxv

Chapter 5: Conclusion..... xxvi

References..... xxvii

Figures

Figure 1. Optimal Production Schedule for Instance A xviii

Figure 2. Optimal Production Schedule for Instance B (Remedy 1) xxii

Tables

Table 1. Characteristics of the benchmark instances viii

Table 2. Mapping between benchmark file sections and parser data structures x

Table 3. Capacity utilization for Instance A xviii

Table 4. Inventory profile for Instance A xix

Table 5. Distribution of IIS constraints by constraint group xx

Table 6. Distribution of IIS constraints across planning periods xx

Table 7. Inventory profile obtained with the additional pre-production period..... xxiv

Table 8. Comparison of the two infeasibility remedies xxv

Chapter 1: Introduction

The Multi-Level Capacitated Lot-Sizing Problem (MLCLSP) is a tactical production-planning problem in which several items, linked by a bill-of-materials, must be produced over a discrete planning horizon on a set of capacitated machines. The decision is how much of each item to manufacture in each period so that all external demand is met while inventory-holding and setup costs are minimized. As a big-bucket model, MLCLSP allows multiple products and multiple setups per period, but it does not look inside a period to decide the order in which lots run on each machine. Sequencing is conventionally left to a separate, lower-level scheduling step.

This separation creates a structural difficulty. If lead times are set to zero, predecessor and successor items may both be assigned to the same period, but the lot sizes the model produces are often impossible to schedule because the predecessor cannot physically finish before the successor needs to start. If lead times are set to at least one period, schedules become feasible but throughput time and work-in-process inflate in proportion to the number of bill-of-materials levels. Neither extreme is realistic for production systems running close to capacity.

The model of Almeder et al. (2015) closes this gap by embedding scheduling decisions directly inside the lot-sizing formulation. Each lot receives a continuous start time within the period, sequence-dependent setup states and changeovers are tracked on every machine, and a synchronization condition guarantees that the in-period inventory of every item stays non-negative at every instant, not only at the period boundary. The paper develops two variants of this idea: a batching formulation in which a successor can begin only after a predecessor's entire lot is finished, and a lot-streaming formulation in which a successor may begin as soon as parts of its predecessor are available, so that production and consumption overlap continuously.

This project implements the lot-streaming variant. The objective is to compute integrated lot sizes and intra-period schedules that are simultaneously feasible and lean on a set of benchmark instances. The implementation is written in Python with the Gurobi solver. It uses the inventory-balance, machine-capacity, setup-sequencing and timing constraints of the paper (constraints 7–14), together with the two synchronization constraints 21 and

22 that form the lot-streaming core. The synchronization conditions involve $\min\{\cdot,\cdot\}$ and positive-part operators; the implementation models these directly through Gurobi's general-constraint machinery rather than through the big-M linearization given in the paper's Appendix A.

Chapter 2: Mathematical Model

This section presents the lot-streaming formulation exactly as implemented in .

The notation follows the paper.

2.1 Sets and Indices

Four index sets are used.

$N = \{1, 2, \ldots, N\}$ (items, including finished goods, subassemblies and raw materials)

$T = \{1, 2, \ldots, T\}$ (planning periods)

$M = \{1, 2, \ldots, M\}$ (machines)

$\phi(m) \subseteq N$ (items that can be produced on machine $m \in M$)

Each item is assigned to exactly one machine, so $\{\phi(m)\}_{m \in M}$ partitions N.

The bill-of-materials defines a successor relation $\Gamma(i)$:

$\Gamma(i) = \{j \in N : a_{ij} > 0\}$ (direct successors of item i)

The typical indices used below are $i, j, k \in N$ for items, $t \in T$ for periods and $m \in M$ for machines.

2.2 Parameters

The parameters are read from the instance file *parser.py* and are summarized in Table 1.

Symbol	**Meaning**
a_{ij}	Units of item i required to produce one unit of item j (gozinto factor)
E_{it}	External demand for item i in period t

h_i	Holding cost per unit of item i per period
c_j	Setup cost charged when a changeover terminates at item j
p_i	Processing time of one unit of item i (normalized, see Section 3)
s_j	Setup time required when a changeover terminates at item j (normalized)
L_m	Original capacity of machine m (time units), constant across periods
$I_{i,0}$	Initial inventory of item i at the start of the horizon

Note on the cost structure: The paper writes setup costs and setup times in fully sequence-dependent form, c_{ij} and s_{ij}. The benchmark instances supply a single value per item, so the implementation attaches the cost and the time to the destination item only, that is $c_{ij} = c_j$ and $s_{ij} = s_j$ for every predecessor i. This is a special case of the paper's formulation.

Note on holding cost: The instance files give one holding cost per item rather than one per (item, period) pair, so h_{it} collapses to h_i.

2.3 Decision Variables

Symbol	**Domain**	**Meaning**
I_{it}	$R \geq 0$	Inventory of item i at the end of period t
X_{it}	$R \geq 0$	Production quantity of item i in period t
μ_{it}^{s}	[0, 1]	Continuous start time of item i's lot in period t (normalized to a unit period)
α_{itm}	{0, 1}	1 if machine m is set up for item i at the beginning of period t
T_{ijtm}	{0, 1}	1 if a changeover from item i to item j happens on machine m during period t

The setup-state variable α_{itm} is defined for the post-horizon boundary as well, i.e. for $t \in \{1, \ldots, T+1\}$, so that the carryover of the setup state out of the last period is a feasible setup configuration. Self-changeovers are forbidden by fixing $T_{iitm} = 0$ for every $i \in \phi(m)$, t, m.

2.4 Objective Function

The objective minimizes the sum of inventory-holding costs and changeover costs:

$$v(F_L) = \min \sum_{i=1}^{N} \sum_{t=1}^{T} h_{it} \cdot I_{it} + \sum_{i=1}^{N} \sum_{j=1}^{N} \sum_{t=1}^{T} \sum_{m=1}^{M} c_{ij} \cdot T_{ijtm}$$

The first term penalizes stock carried from one period to the next; the second term penalizes every setup performed during the horizon. The setup state carried in from period $t-1$ via α_{itm} incurs no cost, so the model rewards setup carryover when possible.

2.5 Constraints

Constraint (7) – Inventory balance

$$I_{it} - I_{i(t-1)} - X_{it} + \sum_{j \in \Gamma(i)} a_{ij} \cdot X_{jt} + E_{it} = 0 \quad i, t$$

Purpose: Conservation of material across periods. The ending inventory of an item equals the inventory carried in, plus what was produced this period, minus what successors consumed via the bill-of-materials, minus the external demand.

Interpretation: Compared with the classical MLCLSP balance, the lead-time index $X_{i,t-l_i}$ is replaced by X_{it}. Production and consumption may share a period, which is precisely what makes lot-streaming meaningful. Initial inventories are passed in via $I_{i,0}$.

Constraint (8) – Production–setup linking

$$p_i \cdot X_{it} \leq \sum_{j \in \phi(m)} T_{jitm} + \alpha_{itm} \quad i \in \phi(m), t, m$$

Purpose: Production of item i on machine m is permitted only if the machine is actually set up for item i during period t, either through a changeover into i or by the setup carried in from the previous period.

Interpretation: Since the period length is normalized to one, $p_i X_{it} \leq 1$ whenever the right-hand side is one. If neither a changeover into i nor a carryover for i is paid, the right-hand side is zero and X_{it} is forced to zero.

Constraint (9) – One setup state per machine

$$\sum_{i \in \phi(m)} \alpha_{itm} = 1 \quad t, m$$

Purpose: At the start of every period, every machine is set up for exactly one item drawn from the set of items it can process. The constraint is enforced for the post-horizon boundary $t = T+1$ as well, so that the setup configuration at the end of the last period is a valid state.

Interpretation: This rules out ambiguous setup states (no machine can be partially or simultaneously prepared for two items) and supplies the boundary condition needed by the flow-conservation constraint that follows.

Constraint (10) – Setup-state flow conservation

$$\sum_{j \in \phi(m)} T_{jitm} + \alpha_{itm} = \sum_{j \in \phi(m)} T_{ijtm} + \alpha_{i(t+1)m} \quad i \in \phi(m), t, m$$

Purpose: This is a flow-conservation equation for the setup state at item i. The amount of setup "flowing in" to item i (either by a changeover or as a carryover from the previous period) equals the amount flowing out (either through a changeover to another item or through a carryover to the next period).

Interpretation: Together with constraint (9), this ties the sequence of changeovers on a machine into a coherent open tour that begins at the initial setup state and ends at the setup state carried into the next period.

Constraint (11) – At most one changeover into each item

$$\sum_{j \in \phi(m)} T_{jitm} \leq 1 \quad i \in \phi(m), t, m$$

Purpose: No item is the destination of more than one changeover within a single period and on a single machine. Together with the big-bucket assumption that an item is produced at most once per period, this means each item has a unique production slot when it is produced at all.

Interpretation: This rules out the kind of split-lot escapes that would let a sequencer recover feasibility by producing the same item twice in one period.

Constraints (12) and (13) – Sequencing and start-time timing

Constraint (12) (consecutive lots, when item *j* is not the carryover into the next period):

$$\mu_{it}^{s} + p_i \cdot X_{it} + s_{ij} \cdot T_{ijtm} + T_{ijtm} - 1 - \alpha_{j(t+1)m} \leq \mu_{jt}^{s}$$
$$i \in \phi(m), j \in \phi(m), t, m$$

Constraint (13) (last product in the period's sequence):

$$\mu_{it}^{s} + p_i \cdot X_{it} + s_{ij} \cdot T_{ijtm} + \alpha_{j(t+1)m} - 1 - \alpha_{jtm} \leq \mu_{jt}^{s}$$
$$i \in \phi(m), j \in \phi(m), t, m$$

Purpose: Both inequalities encode the same scheduling rule, *Start(j)* ≥ *Finish(i)* + *Setup(i* → *j)*, but they activate in two different topological situations. Constraint (12) is the active one when an actual changeover from *i* to *j* takes place on machine *m* during period *t* and *j* is not the item carried over into the next period. Constraint (13) handles the symmetric case in which *j* is the last item produced in the sequence, so that *j*'s start time still has to obey the rule even though the path does not loop back to it.

Interpretation: Together, these inequalities eliminate disconnected subtours (sequences that cycle between items without including the initial setup state) and pin the start times *s* to the finish times of the items immediately preceding them. When neither (12) nor (13) is "active" (no real changeover, or the carryover/last-product conditions are not met), the left-hand sides become non-positive and the constraints are trivially satisfied.

Constraint (14) – Period capacity

$$\mu_{it}^{s} + p_i \cdot X_{it} + \sum_{j \in \phi(m)} s_{ij} \cdot T_{ijtm} \leq 1 \quad i \in \phi(m), t$$

Purpose: The finish time of item *i* plus the setup time of whatever changeover follows it must fit inside the unit-length period.

Interpretation: Period capacity is enforced item by item rather than as a single aggregate sum. Because each machine produces each item at most once per period (constraint 11), this is equivalent to the machine-level capacity bound: total processing plus total setup time fits inside the period.

Constraint (21) – In-period inventory at item *i*'s start

$$I_{i(t-1)} \geq \sum_{j \in \Gamma(i)} a_{ij} \min\left(X_{jt}, \frac{1}{p_j}(\mu_{it}^s - \mu_{jt}^s)^+\right) \quad i, t$$

where $(x)^+ = \max\{x, 0\}$

Purpose: At the instant production of item *i* begins, no units of *i* have yet been produced in the current period, so the entire consumption of *i* by any successor *j* that has already started must be served from the opening stock $I_{i,t-1}$.

Interpretation: The right-hand side is the cumulative consumption of item *i* by all of its successors $j \in \Gamma(i)$ between the moments those successors started and the moment *i* itself starts. For each successor *j*, the inner min caps consumption at the lesser of two quantities: the full lot X_{jt}, or the amount *j* could have produced at its rate $1/p_j$ in the elapsed time. The positive-part operator ensures the term is zero when *j* has not yet started.

Constraint (22) – In-period inventory at each successor's finish

$$I_{i(t-1)} + \min\left\{X_{it}, \tfrac{1}{p_i}(\mu_{kt}^s + p_k X_{kt} - \mu_{it}^s)^+\right\} \geq \sum_{j \in \Gamma(i)} a_{ij} \min\left\{X_{jt}, \tfrac{1}{p_j}(\mu_{kt}^s + p_k X_{kt} - \mu_{jt}^s)^+\right\}$$

Purpose: At the moment any successor *k* of item *i* finishes, the opening stock of *i* plus the amount of *i* that has been produced so far must cover the cumulative consumption of *i* across all of its successors up to that instant.

Interpretation: The left-hand side captures the supply of *i* available by time $\mu_{it}^s + p_k X_{kt}$:

the opening stock, plus production of *i* from its own start up to that instant, capped by the total lot size. The right-hand side captures the cumulative consumption of *i* by all successors $j \in \Gamma(i)$ across the same interval. Theorem 1 in the paper shows that checking the synchronization condition only at item *i*'s own start (constraint 21) and at every successor's finish (constraint 22) is both sufficient and necessary for the in-period inventory of *i* to stay non-negative on the entire interval [0,1]. Together, these two inequalities are what permits the supply of *i* and the consumption of *i* to overlap continuously while preserving feasibility.

Implementation note: Both inequalities contain $\min\{\cdot,\cdot\}$ terms and a positive-part operator, which are non-linear. The implementation models each occurrence directly as a Gurobi general constraint. For each (*i, t*) and each reference time expression, an auxiliary variable captures (ref - μ_{it}^s), a 'GenConstrMax' lifts it to its positive part, the result is

scaled by $1/p_i$, and a 'GenConstrMin' returns $\min\{X_{it}, \text{scaled}\}$. The big-M linearization of Appendix A of the paper is not used.

Chapter 3: Implementation

3.1 Benchmark instances

The computational experiments were conducted using the benchmark instances provided for this project. Four instances were selected:

- A_G001545_MLCLS
- B_G511541_MLCLS
- C_K805132_MLCLS
- D_G819321_MLCLS

The instances differ in the number of items, planning periods, resources, and BOM complexity.

Instance	Items	Periods	Resources	Set up time
A	10	4	3	No
B	10	4	3	Yes
C	40	16	6	No
D	40	16	6	Yes

Table 1. Characteristics of the benchmark instances

Instances A and B represent relatively small planning problems with only ten items and four planning periods.

Instances C and D are substantially larger, containing forty items, sixteen planning periods, and six resources. These instances display a more complex bill-of-material (BOM) structure and generate significantly larger mixed-integer programming models.

Each instance contains the following information:

- Item-specific parameters, including setup costs, holding costs, lead times, and initial inventories.
- A bill-of-material (BOM) matrix describing the precedence relationships between products.
- External demand for each item and period.
- Resource capacities for each machine and period.
- Production and setup capacity requirements.
- Overtime cost parameters.

Instances A and B represent a relatively compact production system with a three-level product structure, consisting of end items, sub-assemblies and low-level items. In contrast, Instances C and D involve a substantially more complex five-level product structure with two independent product families. Demand occurs only at the final products, while all remaining items are produced only for downstream assembly operations. In all instances, no initial inventory is provided, so all material requirements must be satisfied through production within the planning horizon.

Collectively, the four benchmark instances provide a suitable dataset for evaluating both the correctness of the implementation and the scalability of the model under different production planning.

3.2 Data Parsing

The benchmark instances are provided as structured text files in the *.dat* format. Before constructing the model, all instance data must be extracted and converted into data structures that can be processed by Python and Gurobi.

The purpose of the parser is to read the benchmark files and store data in a structured format without modifying the original values.

3.2.1 Section Identification

Each instance file is organized into several sections describing different aspects of the production planning problem, such as item parameters, bill-of-material information, and resource capacities.

The parser identifies each section by locating its corresponding header. Once a header is found, the subsequent rows are read according to the dimensions of the instance. Empty lines are ignored during the reading process.

Examples of section headers include:

- NumberOfPeriods,Items,Resources
- SetupCost,HoldingCost,LeadTime,InitialInventory,NameOfItem
- BOM(c_ij=NumberOfItems_i_NecessaryToProduceItem_j)
- ExternalDemandForEachItemAndPeriod
- CapacityLimitsForEachResourceAndPeriod

This approach allows the parser to read all benchmark instances using the same procedure.

3.2.2 Mapping Between File Sections and Data Structures

The information extracted from the benchmark files is stored in Python lists and matrices. Table 2 summarizes the mapping between the file sections and the corresponding data structures used in the implementation.

File Section	Symbol / Content	Data Type
NumberOfPeriods,Items,Resources	Number of periods (*T*), items (*N*), resources (*M*)	Integers

File Section	Symbol / Content	Data Type
SetupCost,HoldingCost,LeadTime, InitialInventory,NameOfItem	Setup cost, holding cost (h_i), lead time, initial inventory, item name	Array by item
BOM	Gozinto coefficients (a_{ij})	$N \times N$ matrix
ExternalDemand	External demand (E_{it})	$N \times T$ matrix
CapacityLimits	Available capacity (L_{mt})	$M \times T$ matrix
CapacityNeedsForProduction	Processing time requirements (p_{mi})	$M \times N$ matrix
CapacityNeedsForSetup	Setup time requirements (s_{mi})	$M \times N$ matrix
OverTimeCosts	Overtime costs (not used in Lot-streaming model)	One value per resource

3.2.3 Parser Output

After reading all sections, the extracted information is collected into a Python dictionary. This dictionary serves as the input for the preprocessing and model construction stages. This structure provides a centralized representation of all data required by the optimization model.

To verify the correctness of the parser, the extracted data were printed and compared with the original benchmark files. This verification step ensured that all benchmark instances were successfully imported no information was lost during the reading process.

3.3 Data Preprocessing and Set Construction

3.3.1 Capacity Normalization

The formulation proposed by the paper assumes that the capacity of each resource-period is normalized to one unit of available production time. Therefore, the original capacities provided in the benchmark instances were used to scale all processing and setup times.

Let L_m denote the original capacity of resource m. Since this capacity remains constant across all periods ($L_{mt} = L_m$ for all t), the normalized parameters are obtained as

$$p'_{mi} = \frac{p_{mi}}{L_m}$$

$$s'_{mi} = \frac{s_{mi}}{L_m}$$

After normalization, all resource capacities become

$$L_{mt}^{\text{norm}} = 1$$

for every resource m and period t.

The original capacities were stored separately in order to transform the final results back to the original scale.

3.3.2 Parameter and Set Construction

Processing and setting up parameters

The benchmark instances define processing and setup requirements through resource-item matrices.

Since each item can be produced on only one resource, for each item i, the unique resource m for which $p_{mi} > 0$ was identified and used to extract the normalized processing time p_i and setup time s_j.

$$p_i = p_{mi}$$

$$s_j = s_{mj}$$

The paper defines a sequence-dependent setup time s_{ij}, but since the data only provide an item-specific value, it was simplified to s_j.

Construction of Auxiliary Sets

Several auxiliary sets were constructed from the capacity and BOM data to simplify the implement of the formulation.

Resource assignment set $\phi(m)$

For each resource m, the set

$$\phi(m) = \{i \mid p_{mi} > 0\}$$

contains all items that are available to be processed on resource m. Sicne each item is assigned to exactly one resource, these sets define the feasible item–resource assignments that are enforced in the sequencing and setup-state constraints.

Successor set $\Gamma(i)$

For each item i, the set

$$\Gamma(i) = \{j \mid a_{ij} > 0\}$$

contains all direct successors of item i in the bill-of-material (BOM) structure. In other words, item i is required to produce each item $j \in \Gamma(i)$.

3.4 Model Implementation

3.4.1 Decision Variables

The model was implemented in Python using the Gurobi Optimizer. Five decision variables were created. The continuous variables X_{it} represent production quantities, I_{it} represent inventory levels, and μ_{it} represent normalized production start times. In addition, binary setup-state variables α_{itm} and changeover variables T_{ijtm} were introduced to model machine setups and production sequences.

The setup-state variables α_{itm} were defined for $T + 1$ setup states rather than T periods in order to capture the setup state at the end of the last planning period.

3.4.2 Objective Function

The objective function was implemented using Gurobi's *quicksum()* operator. Inventory holding costs were accumulated over all inventory variables I_{it}, while setup costs were associated with the binary changeover variables T_{ijtm}.

3.4.3 Implementation of Problem Constraints

Inventory Balance Constraint (7)

Constraint (7) was implemented directly from the mathematical formulation. In order to fomulate the previous inventory term, the initial inventory parameter was used for first period, while the decision variable $I_{i,t-1}$ represented the incoming inventory for all subsequent periods. The successor set $\Gamma(i)$ was represented through the BOM matrix coefficients a_{ij}.

Production–Setup Linking Constraint (8)

Constraint (8) was implemented for each feasible item–resource pair contained in the set $\phi(m)$. The normalized processing time p_i was used to link production quantities to the setup state variables, ensuring that an item can only be produced on the machine set up for it in that period.

Setup State Constraint (9)

Constraint (9) was implemented for all setup states. The setup-state variables α_{itm} were defined for $T + 1$ setup states in order to capture the machine set up at the end of the planning horizon.

Setup Flow Constraint (10)

Constraint (10) was implemented for each item–resource pair in $\phi(m)$ over all periods, linking the changeover variables T_{ijtm} with the setup states α_{itm} so that the machine configuration is conserved across consecutive periods.

Changeover Constraint (11)

Constraint (11) was implemented for each item–resource pair in $\phi(m)$over all periods, limiting the changeover variables T_{jitm} to at most one changeover into each item per period.

Timing Constraints (12)–(13)

Constraints (12) and (13) were implemented for each item–resource pair in $\phi(m)$ across all periods, using the normalized start times μ_{it}, processing times p_i, and setup times s_j, with (12) covering consecutive items within a period and (13) the item carried into the next period. Since all time-related parameters were normalized during preprocessing, the timing relationships are expressed directly on the interval [0,1].

Capacity Constraint (14)

Constraint (14) was implemented using the normalized processing and setup times. Because the capacity of every resource-period was normalized to one unit, the completion time of each production lot together with the required setup activities must remain within the interval [0,1]

In-period Inventory Constraints (21)–(22)

Constraint (20) contains the term:

$$\min\left(X_{it}, \frac{1}{p_i}(\tau - \mu^s_{it})^+\right)$$

with

$$(\tau - \mu^s_{it})^+ = \max(0, \tau - \mu^s_{it}).$$

This expression is non-linear due to the nested **max** and **min** operators. To model this expression in Gurobi, a helper routine *min_term()* was developed. The positive-part operator is represented by an auxiliary variable together with a general maximum constraint, while a second auxiliary variable is used to represent the minimum between the planned lot size and the available quantity.

For a given item i, period t, and reference time τ (ref_expr), the helper routine *min_term()* builds the term as follows:

1. $z = \tau - \mu_{it}$: a free auxiliary variable capturing the time difference.
2. $z^+ = \max(z, 0)$, enforced by *addGenConstrMax*, yielding the positive part.
3. $\text{scaled} = \frac{1}{p_i} z^+$: a linear scaling converting elapsed time into a producible quantity.

4. term = min (X_{it}, scaled), enforced by addGenConstrMin, capping the produced quantity at the planned lot size X_{it}.

Consequently, the original expression is reformulated into a sequence of linear relations and Gurobi general constraints without altering its mathematical meaning.

Constraints (21) and (22) are obtained directly from the generic implementation of Constraint (20) by substituting different reference times τ. For Constraint (21), $\tau = \mu_{it}$, corresponding to the start time of item i. For Constraint (22), $\tau = \mu_{kt} + p_k X_{kt}$, corresponding to the completion time of successor item k. Therefore, both constraints reuse the same helper function min_term(), differing only in the choice of τ.

Chapter 4: Computational Results

4.1 Computational Results for Instance A

The model was successfully solved to optimality for instance A_G001545_MLCLS. The test instance consists of 10 items, 4 planning periods, and 3 production resources. Gurobi found an optimal solution with a zero optimality gap after 27 seconds of computation.

Solution Summary

Measure	Value
Status	Optimal
Objective value	11,116.25
Holding cost	186.25
Setup cost	10,930.00
Runtime (s)	53.33
MIP Gap	0.00%

Setup costs account for more than 98 % of the total cost, while inventory holding costs contribute only a small fraction of the objective value. This indicates that the optimal production plan maintains relatively low inventory levels while accepting a large number of setup changes.

Production Sequences

Figure 1 illustrates the optimal production schedule obtained for instance **A_G001545_MLCLS.dat** over the four planning periods with each colored block representing a production lot. The item number and production quantity Q are displayed inside each block, while the small grey values shown beneath the blocks represent the normalized start times μ_{it} determined by the model.

For readability, the figure is presented in normalized time units; the corresponding schedule expressed in the original capacity units is provided separately in the result file.

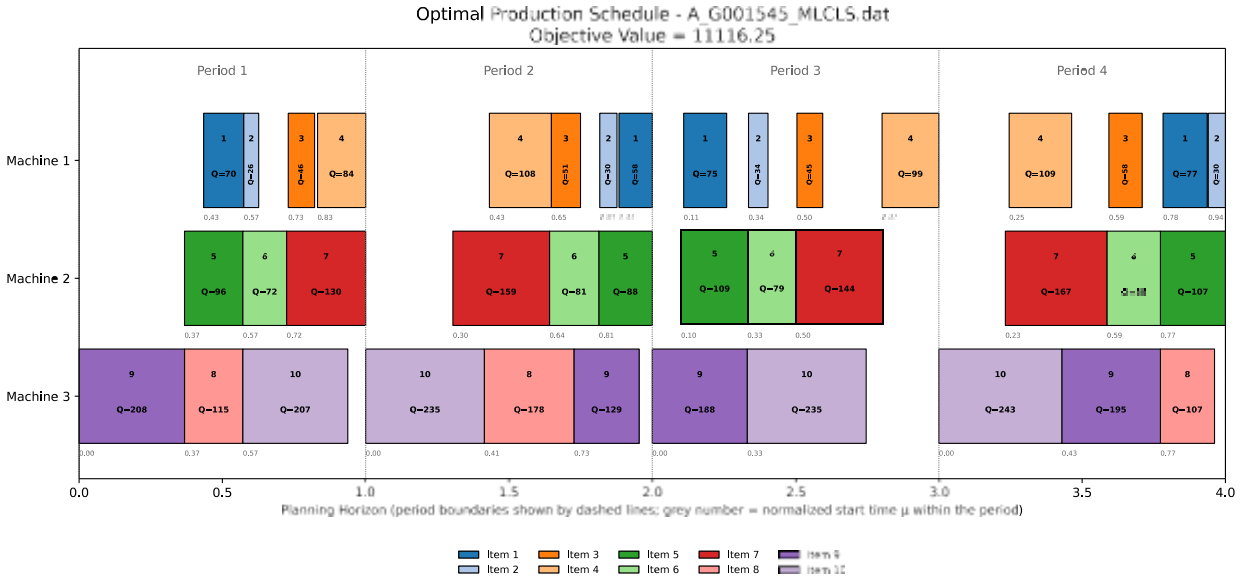


Fig. 1. Optimal Production Schedule for Instance A

The schedule reveals stable production patterns across the planning horizon. Machine 1 alternates between the sequences **1 → 2 → 3 → 4** and **4 → 3 → 2 → 1**, while Machine 2 follows the corresponding patterns **5 → 6 → 7** and **7 → 6 → 5**. Machine 3 produces items 10–8 and exhibits greater flexibility. This pattern results from the **setup-state** and **changeover** decisions. The first item processed in each period is determined by the setup state, and the remaining sequence follows from the changeover decisions.

In this production planning, some lower-level item can start its production simultaneously with its higher-level item, because the model is based on the **lot-streaming concept**. The synchronization constraints require that at every time τ the cumulative consumption of an item by its successors does not exceed the cumulative quantity of that item produced up to τ:

consumption of item i by successors up to time $\tau \leq$ *Available quantity of item i up to time* τ

For example, in Period 1, although item 8 is required to produce item 5, both items start at $\mu \approx 0.37$. This occurs because the synchronization constraints are formulated in cumulative form and do not impose a positive time lag between the start of a component and the start of its successor. At the common start time, both cumulative production and cumulative consumption are zero, so the synchronization constraint is satisfied. Since item 8 is produced at a higher rate (≈ 567 units/time) than it is consumed by item 5 (≈ 472 units/time), the cumulative production of item 8 always stays ahead of the cumulative consumption by item 5. As a result, both items can start production at the same time while maintaining material feasibility.

Capacity Utilization

For Instance A, no setup times are incurred. Therefore, machine utilization is determined solely by production processing times, and the reported capacity utilization reported in **Table 3** corresponds directly to the production load on each resource.

Period	Machine 1	Machine 2	Machine 3
1	45.2%	63.2%	93.8%
2	49.4%	69.6%	95.4%
3	50.6%	70.4%	74.6%
4	54.8%	76.8%	96.2%

Machine 3 exhibits the highest utilization throughout the planning production because it is responsible for producing the lowest-level BOM components (Items 8, 9, and 10), which are required by multiple successor items. Since no initial inventory is available, these components must be produced before or concurrently with the production of higher-level items. Consequently, Machine 3 carries a substantial workload and operates close to its capacity limit in most periods.

Inventory Levels

Only three items carry positive inventory between periods.

Item	Period 1	Period 2	Period 3	Period 4
8	0.00	109.00	0.00	0.00
9	40.47	0.00	11.95	0.00
10	24.83	0.00	0.00	0.00

Table 4. Inventory profile for Instance A

Inventory is generally kept low throughout the planning horizon. Positive inventories are generated only for lowest-level BOM components (Items 8, 9, and 10), and are carried for at most one period before being consumed in subsequent production stages. This observation is consistent with the relatively small contribution of inventory holding costs to the objective function.

4.2 Computational Results for Instance B

4.2.1 Analysis of infeasibility

For instance B_G511541_MLCLS, no feasible solution was found. An IIS (Irreducible Infeasible Subsystem) analysis was performed to investigate the source of infeasibility.

The IIS analysis reveals that the setup structure, timing constraints, and in-period inventory constraints are the main sources of infeasibility, while the inventory balance and capacity constraints play only a minor role.

Constraint Group	In IIS	Count
Inventory balance (7)	Yes	8
Setup structure (8)–(11)	Yes	31
Timing (12)–(13)	Yes	34
Capacity (14)	Yes	5
In-period inventory (21)–(22)	Yes	42

Total	120

Table 5. Distribution of IIS constraints by constraint group.

A period-wise analysis shows that most IIS constraints are concentrated in the first planning period.

Period	IIS Constraints
Period 1	109
Period 2	7
No period index	4
Total	120

Table 6. Distribution of IIS constraints across planning periods.

This indicates that infeasibility originates primarily in **Period 1**. Since all initial inventories are zero, the first-period demand and BOM requirements must be satified entirely through production within the same period. As a result, sufficient component quantities cannot be made available early enough to support downstream production, causing the infeasibility of the instance. Additional evidence supporting this explanation is provided in Remedy 1, where the introduction of a pre-production period restores feasibility.

4.2.2 Remedies for Infeasibility

Remedy 1: Additional Pre-production Period

To overcome the infeasibility, an additional planning period with zero external demand was introduced before the original planning horizon. The added period is used to pre-produce critical low-level components and build inventory before customer demand occurs.

Solution Summary

Measure	Value
Status	Optimal
Objective value	**8,414.21**
Holding cost	2,769.21
Setup cost	5,645.00
Runtime (s)	53.33
MIP Gap	0.00%

The optimal solution yields a total cost of 8,414.21. Setup costs account for 5,645.00 (67.1% of the objective value), while holding costs contribute 2,769.21 (32.9%). Setup costs are dominant due to the pre-production period, which which necessitates additional production activities and changeovers.

Production Schedule

Figure 2 presents the optimal production schedule for the modified instance **B_G511541_MLCLS**, where an additional planning period with zero external demand was added. Production blocks display the item number and production quantity. Grey hatched segments denote setup time for a changover, whereas the grey values beneath the blocks correspond to the normalized start times μ_{it}. For readability, the figure is presented in normalized time units; the corresponding schedule expressed in the original capacity units is provided separately in the result file.

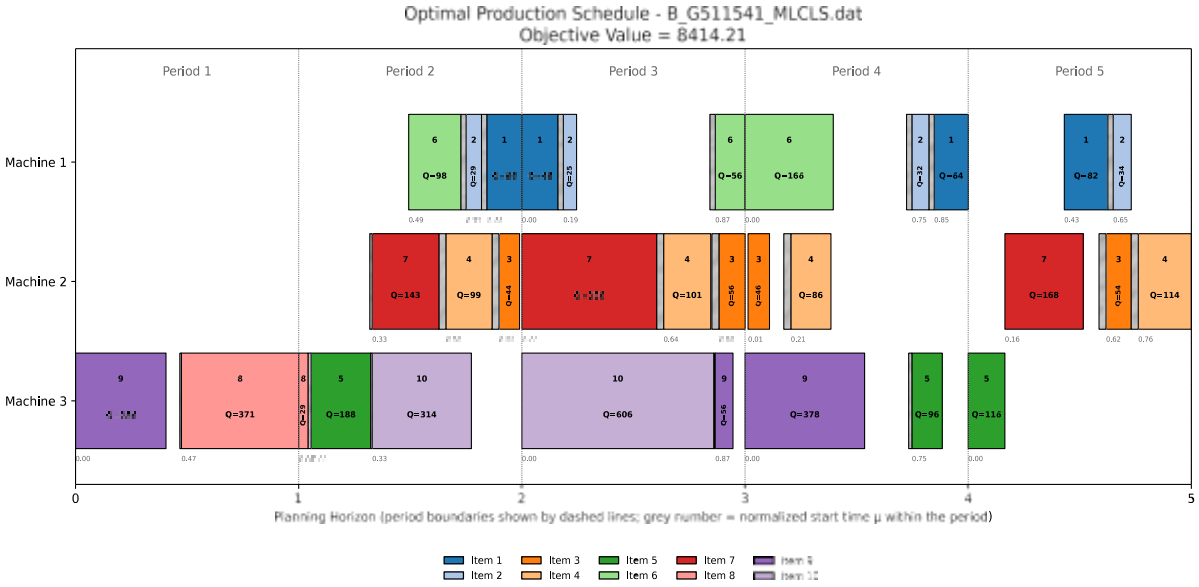


Fig. 2. Optimal Production Schedule for Remendy 1

The production sequences are less repetitive than in Instance A due to the additional pre-production period. On Machine 1, production alternates between the sequences 6 → 2 → 1and 1 → 2, while Machine 2 primarily follows the patterns 7 → 4 → 3 and 4 → 3. Machine 3 exhibits the greatest flexibility, producing different combinations of Items 5, 8, 9, and 10 across periods. As in Instance A, the first item processed in each period is determined by the setup state, while the remaining sequence is governed by the changeover decisions.

As shown, Machine 3 is heavily utilized during the additional period to produce **370.69 units of Item 8** and **286.00 units of Item 9**, which are low-level BOM components required for the production of downstream items. The resulting inventories are carried over to the following periods and consumed accordingly, allowing the model to satisfy all material requirements without violating lot-streaming constraints.

From the added period onward, all three machines become active and operate according to the expected multi-level BOM structure. Specifically, Machine 3 produces the low-level components (Items 5, 8, 9, and 10), Machine 2 manufactures the sub-assemblies (Items 3, 4, and 7), and Machine 1 produces the end items (Items 1, 2, and 6).

The schedule provides additional insight into the source of infeasibility identified in the original instance. Items 1 and 2, which have external demand in the first period, require Item 5 according to the BOM structure. In turn, Item 5 requires both Items 8 and 9

according to the BOM structure. Consequently, the production of Items 8, 9, and 5 must occur sequentially on the same machine, which cannot be completed within a single period, causing the infeasibility of the original instance.

Setup activities are considered when scheduling production operations. Nevertheless, the additional pre-production period provides sufficient flexibility to accommodate both setup activities and material requirements, resulting in a feasible schedule with an objective value of **8,414.21**.

Capacity Utilization

The capacity utilization obtained from the optimal solution is summarized in Table 5.

Period	Machine 1	Machine 2	Machine 3
1	0.0%	0.0%	93.1%
2	46.0%	59.8%	75.2%
3	35.5%	93.2%	93.9%
4	62.4%	27.6%	67.2%
5	27.6%	70.2%	16.4%

The additional pre-production period is almost entirely dedicated to Machine 3, which operates at **93.1%** utilization while Machines 1 and 2 remain idle. This confirms that the original infeasibility was primarily caused by insufficient time to produce the required low-level components before higher-level production had to begin.

Inventory Levels

Positive inventories are observed for several items throughout the planning horizon, as shown in Table X.

Item	Period 1	Period 2	Period 3	Period 4	Period 5
5	0.00	93.00	0.00	0.00	0.00
6	0.00	25.00	0.00	88.00	0.00
7	0.00	0.00	132.00	0.00	0.00

Item	Period 1	Period 2	Period 3	Period 4	Period 5
8	370.69	212.00	212.00	116.00	0.00
9	286.00	0.00	0.00	116.00	0.00
10	0.00	72.52	334.00	168.00	0.00

Table 7. Inventory profile obtained with the additional pre-production period.

Unlike Instance A, inventory plays a crucial role in achieving feasibility. Lowest-level items, items 8 and 9, are produced in large quantities during the additional pre-production period and carried forward for future consumption higher-level items.

The inventory table clearly demonstrates the role of the added period. Without the possibility of producing and storing these components before demand occurs, the original instance cannot satisfy all material and timing requirements simultaneously. The modified instance therefore confirms that infeasibility was caused by insufficient preparation time for component production.

Remedy 2: Additional Initial Inventory

The results of Remedy 1 suggest that the infeasibility of Instance B is caused by insufficient component availability at the first period, combined with the concentration of several dependent items on the same machine.

In Remedy 2, the inventories manufactured during the pre-production period in Remedy 1 were added directly to the initial inventory parameters. Specifically, **370.69 units** of Item 8 and **286.00 units** of Item 9 were provided as initial inventory, while the original four-period planning horizon was retained.

Measure	Remedy 1	Remedy 2
Planning Periods	5	4
Feasible	Yes	Yes
Objective Value	8,414.21	6,957.52
Holding Cost	2,769.21	2,112.52

Measure	Remedy 1	Remedy 2
Setup/Changeover Cost	5,645.00	4,845.00
Runtime (s)	15.90	3.79

Table 8. Comparison of the results obtained from the two remedies.

Both approaches successfully eliminate infeasibility by ensuring the availability of critical low-level components before external demand occurs. However, Remedy 2 achieves a lower objective value while preserving the original four-period planning horizon.

Among the two remedies considered, the additional initial inventory approach is the more effective and economical solution. Compared with Remedy 1, the objective value decreases by approximately 17.3%. This improvement arises because Remedy 2 provides the required low-level components as initial inventory at the beginning of the planning horizon, thereby eliminating additional setup and holding costs associated with the extra pre-production period.

The superior performance of Remedy 2 confirms that providing the required components at the start of the horizon is sufficient to restore feasibility and improve overall solution quality.

4.3 Computational Results for Instances C and D

For Instances C and D, the solver was unable to obtain a conclusive result within the prescribed runtime limit. Consequently, neither feasibility nor infeasibility could be formally established.

Compared with Instances A and B, these instances are substantially larger, involving 40 items, 6 machines, and 16 planning periods. The resulting increase in the number of decision variables and constraints significantly enlarges the search space and makes the solution process computationally demanding. Therefore, no production schedule or further analysis can be reported within the limited runtime.

Chapter 5: Conclusion

This project implemented the lot-streaming formulation of the Multi-Level Capacitated Lot-Sizing Problem (MLCLSP) proposed by Almeder et al. (2015). The model integrates production planning and scheduling decisions by simultaneously determining production quantities, inventory levels, machine setup states, changeovers, and production start times. In addition to the classical lot-sizing constraints, synchronization constraints were incorporated to ensure that material consumption never exceeds material availability during production. The benchmark instances were successfully parsed, preprocessed, and transformed into the parameters required by the optimization model. Capacity normalization was applied to satisfy the assumptions of the formulation, and auxiliary sets describing machine assignments and bill-of-material relationships were generated automatically from the input data.

Computational experiments demonstrated that the implementation is capable of producing feasible and optimal solutions for smaller benchmark instances. For Instance A, the model generated an optimal production plan with low inventory levels and efficient machine utilization. For Instance B, the original formulation proved infeasible due to insufficient availability of low-level components during the first planning period. Two alternative remedies were investigated, namely the introduction of an additional pre-production period and the provision of initial inventories. Both approaches restored feasibility, while the initial inventory approach produced the lowest overall cost.

The computational study also highlighted the increasing complexity of synchronization-based lot-sizing models. While smaller instances could be solved to optimality, the larger benchmark instances could not be fully resolved within the available computational limits. This illustrates the significant increase in model size and solution difficulty caused by the integration of detailed scheduling and synchronization decisions.

Overall, the results confirm the advantages of the lot-streaming approach. By explicitly considering production timing and material synchronization, the model generates production plans that are both operationally feasible and more realistic than those obtained from traditional lot-sizing formulations that ignore intra-period scheduling effects.

References

1. Almeder, C., Klabjan, D., Traxler, R., & Almada-Lobo, B. (2015). *Lead time considerations for the multi-level capacitated lot-sizing problem*. *European Journal of Operational Research, 241*(3), 727–738. DOI: https://doi.org/10.1016/j.ejor.2014.08.041
2. Gurobi Optimization, LLC. (2025). *Gurobi Optimizer Reference Manual*. Available at: https://www.gurobi.com/documentation/
3. Williams, H. P. (2013). *Model Building in Mathematical Programming* (5th ed.). John Wiley & Sons. Available at: https://www.wiley.com/en-us/Model+Building+in+Mathematical+Programming%2C+5th+Edition-p-9781118443330